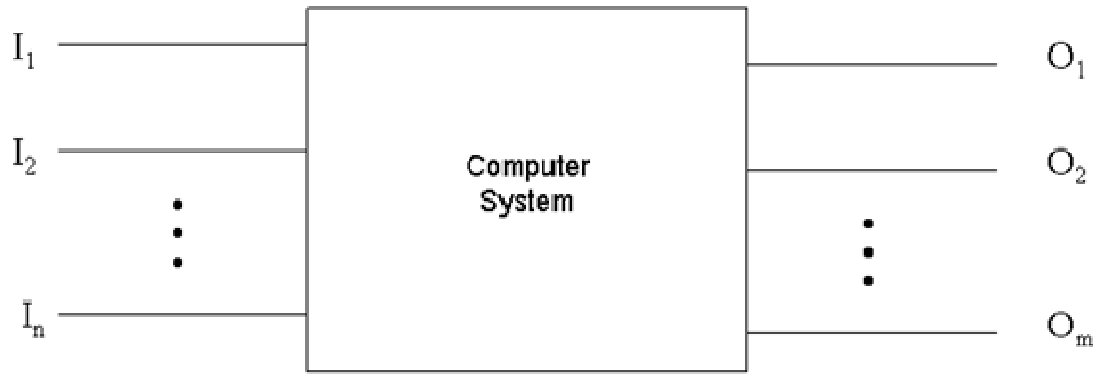
Chapter 1
Real-Time System: Concepts

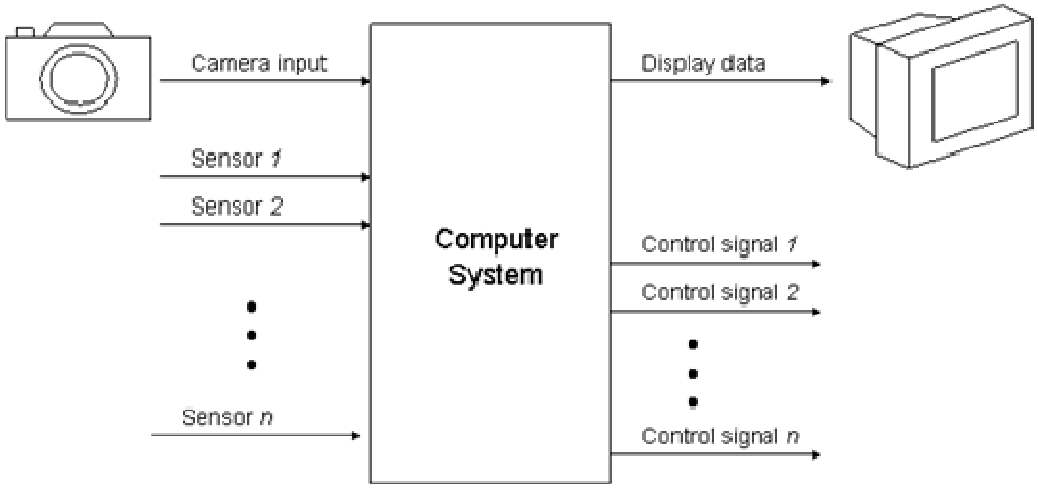
1.1 Terminologies:

1.1.1 System Concepts

Definition: A system is a mapping of a set of inputs into a set of outputs.



A system with *n* inputs and *m* outputs.



Typical real-time control system including inputs from sensors and imaging devices and producing control signals and display information.

Definition: The time between the presentation of a set of inputs to a system (stimulus) and the realization of the required behavior, including the availability of all associated outputs, (response) is called the response time of the system.

1.1.2 Real Time Definition

Definition: *A real-time system is a system that must satisfy explicit (bounded) response-time constraints or risk severe consequences, including failure.*

Definition: *A failed system is a system that cannot satisfy one or more of the requirements stipulated in the formal system specification.*

Definition: *A real-time system is one whose logical correctness is based on both the correctness of the outputs and their timeliness.*

Definition: *A real-time system is one whose logical correctness is based on the correctness of the output and their timeliness. This means, a system that gives correct results but out of deadline is considered a failure.*

Definition: *A real-time system is defined as those systems in which correctness of system depends not only on the logical result of computation but also on time at which results are produced.*

Real-time systems are often reactive or embedded systems. Reactive systems are those that have an ongoing interaction with their environment-for example, a fire-control system that constantly reacts to buttons pressed by a pilot. Embedded systems are those used to control specialized hardware in which the computer system is installed. For example, the microprocessor system used to control the fuel/air mixture in the carburetor of many automobiles is clearly embedded.

Depending upon temporal behavior of Real-Time System they can be characterized as hard, soft and firm real-time system.

Hard Real Time System:

A system crashes if the deadline is not meet by the tasks.

A hard deadline is imposed on a job as a late result produced by the job after the deadline have disastrous consequences.

A hard real-time system is one in which failure to meet a single deadline may lead to complete and catastrophic system failure.

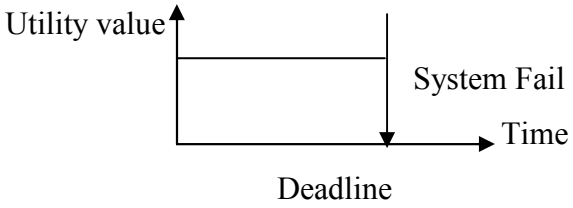


Figure 1.1: Hard Real Time System

For example: Rocket Launching System

Soft Real Time System:

Those real-time system where missing a deadline does not create the system failure but degrades the system performance are called soft real-time systems. Generally, system with no strict deadlines can be thought of as soft real-time system.

A soft real-time system is one in which performance is degraded but not destroyed by failure to meet response-time constraints.

Here utility value becomes less/drops with time.

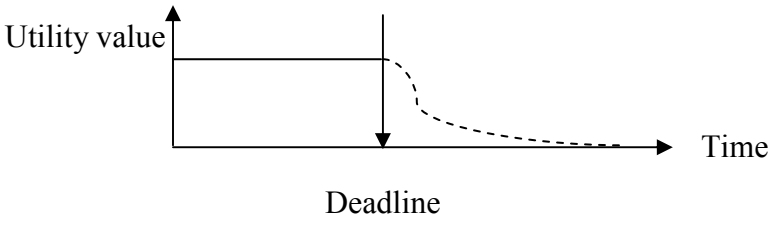


Figure 1.1: Soft Real Time System

For example: Exam Hall

Delay by 10 or 20 minutes allowed. More delay less you will be able to write.

Firm Real Time System:

Those system where the missing of deadline does not cause the system to crash but causes the result to be omitted are called firm-real time system.

System doesn't fail but throws the results (ignores) in case of failure.

A firm real-time system is one in which a few missed deadlines will not lead to total failure, but missing more than a few may lead to complete and catastrophic system failure.

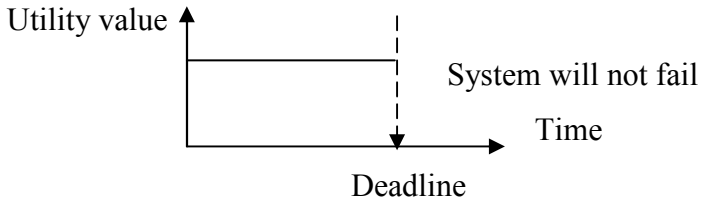
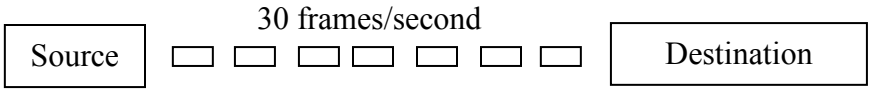


Figure 1.3: Firm Real-Time System

For example: Video Conferencing



During video conferencing, missing a few frames won't cause a disaster.

It is considered in system with hard deadlines but the probability of failure is fairly low.

	Real-time Classification	Explanation
Automated teller machine	soft	Missing even many deadlines will not lead to catastrophic failure, only degraded performance.
Embedded navigation controller for autonomous robot weed killer	firm	Missing critical navigation deadlines causes the robot to veer hopelessly out of control and damage crops.
Avionics weapons delivery system in which pressing a button launches an air-to-air missile	hard	Missing the deadline to launch the missile within a specified time after pressing the button can cause the target to be missed – which will result in catastrophe.

A sampling of hard, soft, and firm real-time systems.

Embedded system

A software system that is completely encapsulated by the hardware that it controls. For example: A cars fuel injection system, a lift control system.

Organic system

A software system that is not highly dependent on the hardware on which it runs and that includes a generalized user interface. For example: An airline reservation system, a library catalogue system.

Semi-detached system

A software system that displays characteristics of both embedded and organic systems. For example: A production line quality monitoring system.

Parts of Typical Real-Time System

- *Controlling System*
  - Computer system acquires information about the environment through sensors
  - Performs computations on data
  - Activates the actuators through some controls
- *Controlled System*
- *Operating Environment*

Example – Car Driver

- *Mission*
  - Reach destination safely
- *Controlled System*
  - Car
- *Operating Environment*
  - Road Conditions and other cars
- *Controlling System Sensors*
  - Human driver: eyes and ears
  - Computer: Camera, Infrared receiver, Laser telemeter
- *Controls*
  - Accelerator, Steering wheel, Brake Pedal
- *Actuators*
  - Wheels, Engine, Brakes

- *Critical Tasks*
  - Steering and Braking
- *Non-critical Tasks*
  - Turning on the radio
- *Performance*
  - A measure of the goodness of outcome, relative to the best outcome possible under the given circumstances
- *Reliability (of driver)*
  - Fault-tolerance is a must

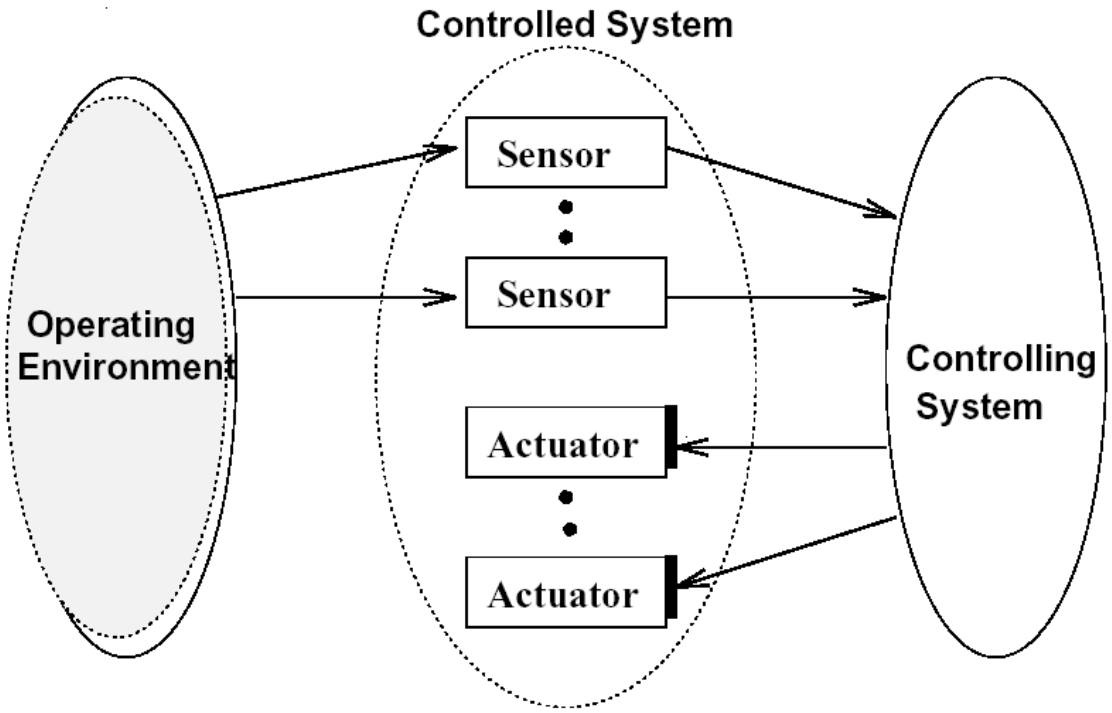


Figure 1.4: Typical Real-Time System Parts

Tasks and Jobs

- Jobs are units of work that are scheduled and executed by the systems.
- The set of related jobs that can be solved by the same algorithm are called a task.
  - A job is an instance of a task.
  - Use of "instance" is same as in complexity theory

Type of Deadlines

A deadline *d* is a time at which a task/job must be complete.

- A *hard deadline* means that it is vital for the safety that this deadline always be met.
- A *soft deadline* means that it is desirable to finish executing the task/job by the deadline, but that no catastrophe occurs if completion is late
  - Some soft tasks only require that the task be completed as soon as possible.
- A *firm deadline* means there is no value in completing the task after its deadline.

Response Time

The time between the presentation of a set of inputs to a system and the realization of the required behavior, including the availability of all associated outputs, is called the response time of the system.

Release Time

The release time is the time at which an instance of a scheduled task is ready to run, and is generally associated with an interrupt.

1.1.3 Types of Real Time Tasks

Periodic Tasks:

Periodic Tasks are activated regularly at fixed rates. Jobs repeated at regular or semi-regular intervals modeled as periodic.

- These tasks are time-driven
- Example: Monitoring temperature of a patient
- The length of time between to successive activations is called the *period*.
- In many practical cases, a periodic task can be characterized by its computation time C and its deadline D.
  - Often the deadline is set equal to the period.
- Consists of a sequence of identical "jobs" or "instances" of the task.

Aperiodic Tasks:

Aperiodic tasks are tasks which are activated irregularly at some unknown and possibly unbounded rate. Jobs have soft or no deadlines. Want responsiveness.

- Event driven
- Example – Activated when condition of patient changes.

Sporadic Tasks:

Sporadic tasks are tasks that are activated with some known and bounded rate. Jobs have hard deadlines

- Necessary to bound the workload generated by such tasks.

	Periodic	**Aperiodic**	**Sporadic**
Synchronous	Cyclic Code Processes scheduled by internal clock	Typical branch instruction Garbage collection	Branch instruction e.g. error recovery Traps
Asynchronous	Clock generated interrupt	Regular, but not fixed period interrupt	Externally generated exception "Random events"

Taxonomy of events and some examples.

1.1.4 Events and Determinism

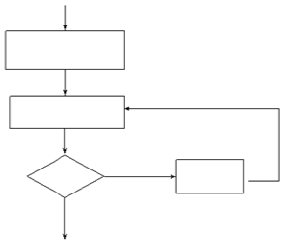
Events

Any occurrence that causes the program counter to change non-sequentially is considered a change of flow-of-control, and thus an event.

Change in Flow of Control

- A change in state results in a change in the flow-of-control
- The decision block suggests that the program flow can take alternative paths
- case, if-then, and while statements represent a possible change in flow-of-control

- Invocation of procedures in Ada and C represent changes in flow-of-control
- In C++ and Java, instantiation of an object or the invocation of a method causes the change in flow-of-control



Taxonomy of Events

An event can be either synchronous or asynchronous:

- *Synchronous* events are those that occur at predictable times in the flow-of-control
- *Asynchronous* events occur at unpredictable points in the flow-of-control and are usually caused by external sources

Moreover, events can be periodic, aperiodic or sporadic

- A real-time clock that pulses regularly is a *periodic* event.
- Events that do not occur at regular periods are called *aperiodic.*
- Aperiodic events that tend to occur very infrequently are called *sporadic.*

Example: Various Types of Events

Type	Periodic	Aperiodic	Sporadic
Synchronous	**Cyclic code**	**Conditional branch**	**Divide-by-zero (trap) interrupt**
Asynchronous	**Clock interrupt**	**Regular, but not fixed-period interrupt**	**Power-loss alarm**

Deterministic Systems

A system is deterministic, if for each possible state and each set of inputs, a unique set of outputs and next state of the system can be determined.

- For any physical system, certain states exist under which the system is considered to be out of control.
- The software controlling such a system must therefore avoid these states.
- In embedded real-time systems, maintaining overall control is extremely important.
- Software control of any real-time system and associated hardware is maintained when the next state of the system, given the current state and a set of inputs, is predictable.
- *Event determinism* means the next states and outputs of a system are known for each set of inputs that trigger events.
- Thus, a system that is deterministic is also event deterministic.
- However, event determinism may not imply determinism.
- While it is a significant challenge to design systems that are completely event deterministic, it is possible to inadvertently end up with a system that is non-deterministic.
- Finally, if in a deterministic system the response time for each set of outputs is known, then, the system also exhibits *temporal determinism.*
- A side benefit of designing deterministic systems is that guarantees can be given that the system will be able to respond at any time, and in the case of temporally deterministic systems, when they will respond.

1.1.5 CPU Utilization

The (CPU) utilization or time-loading factor is a measure of the percentage of non-idle processing.

- The final term to be defined is a critical measure of real-time system performance.
- Because the CPU continues to execute instructions as long as power is applied, it will more or less frequently execute instructions that are not related to the fulfillment of a specific deadline.
- The measure of the relative time spent doing non-idle processing indicates how much real-time processing is occurring.

Utilization (%)	Zone Type	Typical Application
0-25	significant excess processing power – CPU may be more powerful than necessary	various
26-50	very safe	various
51-68	safe	various
69	theoretical limit	embedded systems
70-82	questionable	embedded systems
83-99	dangerous	embedded systems
100+	overload	stressed systems

CPU utilization zones and typical applications and recommendations.

Determining CPU Utilization

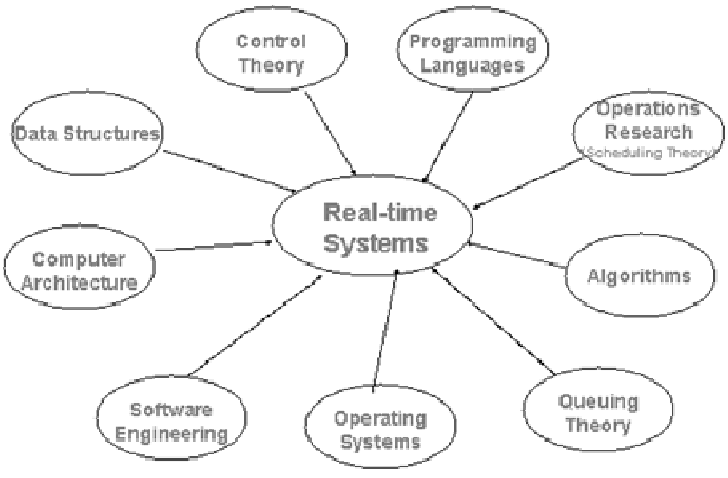
Suppose a system has $n \geq 1$ periodic tasks, each with an execution period of p_i and hence execution frequency $f_i = 1/p_i$. If task i is known to have (or has been estimated to have) a maximum (worst case) execution time of e_i then the utilization factor u_i for task $_i$ is

$$u_i = e_i / p_i \tag{1.1}$$

Then the overall system utilization is

$$U = \sum_{i=1}^{n} u_i = \sum_{i=1}^{n} e_i / p_i \tag{1.2}$$

1.2 Real-Time System Design Issues



Disciplines that impact real-time systems.

- ❖ The selection of hardware and system software, and evaluation of the trade-off needed for a competitive solution.
- ❖ Specification and design of real-time systems, as well as correct and inclusive representation of temporal behavior.
- ❖ Understanding the nuances of programming language(s) and the real-time implications resulting from their compilation.
- ❖ Optimizing (with application-specific objectives) of system fault tolerance and reliability through careful design and analysis.
- ❖ The design and administration of adequate tests, and the selection of appropriate development tools and test equipment.
- ❖ Taking full advantage of open systems technology and interoperability.
- ❖ Estimating and measuring response times and (if needed) reducing them; performing a schedulability analysis.

1.3 Examples of Real Time Systems

Domain	**Applications**
Avionics	• Navigation • Displays
Multimedia	• Games • Simulators
Medicine	• Robot surgery • Remote surgery • Medical imaging
Industrial Systems	• Robotic assembly lines • Automated inspection
Civilian	• Elevator control • Automotive systems

Real-time application domains.

- Aircraft inertial measurement system
- Nuclear plant control
- Airline reservation system
- Pasta sauce bottling plant
- Traffic light control system for 4-way intersection

1.4 Common Misconceptions regarding Real-Time Systems

- *Real-time systems are synonymous with "fast" systems.*

  Many (but not all) hard real-time systems deal with deadlines in the tens of milliseconds.

- *There are universal, widely accepted methodologies for real-time systems specification and design*

  There is still no methodology available that answers all of the challenges of real-time specification and design all the time and for all applications.

- *There is no more a need to build a real-time operating system, because many commercial products exist.*

  Commercial solutions have certainly their place, but choosing when to use an off-the-shelf solution and choosing the right one are continuing challenges.

- *Rate-monotonic analysis has solved "the real-time problem"*

  While rate-monotonic systems provide guidance in the design of real-time systems, and while there is theory surrounding them, they are not a panacea.

- *The study of real-time systems is mostly about scheduling theory.*

  While it is scholarly to study scheduling theory, most published results require impractical simplifications and clairvoyance in order to make the theory work.

1.5 Historical Aspects of Real-Time Systems

Brief history

Year	Landmark	Developer	Development	Innovations
1947	Whirlwind	MIT/US Navy	Flight simulator	Ferrite core memory, "real response times"
1957	SAGE	IBM	Air defense	Specifically designed for real-time
1958	Scientific 1103A	Univac	General purpose	Hardware interrupt
1959	SABRE	IBM	Airline reservation	Hub-go-ahead policy
1962	Basic Executive	IBM	General purpose	First real-time executive
1963	Basic Executive II	IBM	General purpose	Diverse real-time scheduling, Disk resident user/systems programs
1970s	RSX, RTE	DEC, HP	Real-time operating systems	Hosted by mini-computers
1973	Rate-monotonic system	Liu and Layland	Theory	Stated upper bound on utilization for schedulable systems
1980s	RMX-80, MROS 68K, VRTX, etc.	Various	Real-time operating system	Hosted by microprocessors
1983	Ada 83	US Department of Defense	Programming language	Intended for mission critical, embedded, real-time systems
1995	Ada 95	Community	Programming Language	Refinement to Ada 83

1.6 Brief Survey of Real-Time Programming Languages

Ada 95

- Ada (1983) was originally intended to be the mandatory language for all U.S. Department of Defense projects, which included many embedded real-time systems.
- Ada 83, had significant problems and Ada (1995) was introduced to deal with these problems.
- Ada 95 was the first internationally standardized object-oriented programming language.

- Ada 95 is both procedural and object-oriented depending on how the programmer uses it.
- Ada 95 includes extensions to help deal with scheduling, resource contention, and synchronization.
- Other language extensions made Ada 95 an OO language (tagged types, packages, and protected units).
- Ada has never lived up to its promise of universality because users have found it to be too bulky and inefficient.

C

- Invented around 1971, is a good language for "low-level" programming.
- C provides special variable types such as register, volatile, static, and constant, which allows for control of code generation at the high-order language level.
- C supports call-by-value only – call-by-reference can be implemented by passing a pointer to anything as a value.
- Variables declared as type volatile are not optimized by the compiler. This is useful in handling memory-mapped I/O and other instances where the code should not be optimized.
- Automatic coercion (implicit casting of data types) can lead to mysterious problems.
- C provides for exception handling through the use of signals and setjmp and longjmp, constructs to allow a procedure to return quickly from a deep level of nesting.
- Overall C is good for embedded programming – it provides for structure and flexibility without complex language restrictions.

C++

- C++ is a hybrid OO language that was originally implemented as a macro-extension of C.
- Exhibits all characteristics of an OO language.
- C++ extends C functions with classes and class methods, which are functions that are connected to classes.

- C++ still allows for low-level control using inline methods rather than a runtime call.
- The C++ preprocessor can lead to unnecessary complexity. Preprocessors also do weak type checking and validation.
- Misuse of pointers causes many bugs in C/C++ programming – standard libraries of dynamic data structures (e.g. the Standard Template Language) help minimize pointer problems.
- Multiple inheritance allows a class to be derived from multiple parent classes.
- Multiple inheritance is powerful, but it is difficult to use correctly, is subsumed by composition, and is very complicated to implement.
- C++ does not provide automatic GC, which mean– dynamic memory must be managed manually or GC must be implemented.
- Tendency to take existing C code and objectify it by wrapping the procedural code– very bad because brings all the disadvantages of C++ and none of the benefits.
- Bad C++ programmers use a mixture of function and methods leading to confusing programs.
- C versus C++ – is a tradeoff between a "lean and mean" C program and a slower and unpredictable C++ program that will be easier to maintain.

C#

- C# is a C++-like language that along with its operating environment, has similarities to the Java and JVM.
- C# is associated with Microsoft's .NET Framework for scaled down operating systems like Windows CE 3.0.
- C# and the .NET platform may not be appropriate for hard real-time systems – unbounded execution of its GC environment and lack of threading constructs to adequately support schedulability and determinism.
- C# and .NET might be appropriate for some soft and firm real-time systems.

Fortran

- Fortran is the oldest high-order language still used in modern real-time systems (developed circa 1955).

- Old versions of Fortran did not support re-entrant code.
- Fortran was developed in an era when efficient code was essential to optimizing performance in small, slow machines.
- Fortran is weakly typed, but it still can be used to design highly structured code.
- Fortran has no built-in exception handling or abstract data types.
- Fortran is still used in many legacy and even new real-time applications (A new version of Visual Fortran was recently released for .NET).
- Embedded systems written in Fortran typically include a large portion of assembly language code to handle interrupts and scheduling, and communication with external devices.

Java

- Java is an OO language that looks very much like C++.
- Java is interpreted – the code compiles into machine-independent code which runs in a managed execution environment.
- This environment is the Java Virtual Machine, which executes "object" code instructions as a series of program directives.
- Java code can run on any device that implements the virtual machine – "write once, run anywhere."
- There are native-code Java compilers – byte code executes directly "on the bare metal".
- Like C, Java supports call-by-value, but the value is the reference to an object, which appears as call-by-reference for all objects.
- One of the best-known problems with Java involves its garbage collection utility.

Real-time Java

- National Institute of Standards and Technology (NIST) task force was charged with developing a version of Java that was suitable for embedded real-time applications.
- The final report (1999), defines core requirements for the real-time specification for Java (RTSJ).
    - Any garbage collection that is provided shall have a bounded preemption latency.

- Defines the relationships among real-time Java threads.
- Include APIs to allow communication and synchronization between Java and non-Java tasks.
- Includes handling of both internal and external asynchronous events.
- Incorporates some form of asynchronous thread termination.
- Provides mechanisms for enforcing mutual exclusion without blocking.
- Provides a mechanism to allow code to query whether it is running under a real-time Java thread or a non-real-time Java thread.
- Defines the relationships that exist between real-time Java and non-real-time Java threads.

Occam 2

- Occam 2 is based on the Communicating Sequential Processes (CSP) formalism.
- The basic entity in Occam 2 is the process – there are four fundamental types, assignment, input, output, and wait.
- More complex processes are constructed from these by specifying sequential or parallel execution or by associating a process with an input from a channel.
- The Occam 2 language was designed to support concurrency on transputers but compilers are available for other architectures.
- Occam-2 has found some practical implementation in the U.K.

Special real-time languages

- Several specialized languages for real-time have appeared and disappeared over the last 30 years.
- **PEARL:** Process and Experiment Automation real-time Language developed in the early 1970s. Has fairly wide application in Germany, especially in industrial controls settings.
- **Real-time Euclid:** An experimental language that is completely suited for schedulability analysis. This is achieved through language restriction.
- **Real-time C:** A generic name for any of a variety of C macroextension packages. These macroextensions typically provide timing and control constructs that are not found in standard C.

- **Real-time C++:** A generic name for one of several object class libraries specifically developed for C++. These libraries augment standard C++ to provide an increased level of timing and control.
- Others include MACH, Eiffel, MARUTI, ESTEREL.
- Most of these languages are used for highly specialized applications or in research only.